DONE BY : JASMIN NASSER

ID:202301948

sky news عربية

FINAL PROJECT

UNDER SUPERVISION OF:

DR.Mohamed Maher

Search Engine: Sky newsArabia

مدينة زويل
ZEWAIL CITY
ESTABLISHED 2000
INAUGURATED 2011

مدينة زويل للعلوم والتكنولوجيا
Zewail City of Science and Technology

1 INTRODUCTION

Search engines that work fast play an important role in finding information on the internet when a lot of it is available at a person's disposal. In this research, we look into how customizing search engines was done from start to finish. The aim of this project through the use of Python programming, web scraping among other terms related to network analysis remains; which is building such an application with an eye on target population features while providing details concerning challenges encountered during the development process.

2 PROBLEMS THAT I ENCOUNTERED

I. Data Scraping Challenges

Initially, retrieving data from search engine results posed a significant hurdle. Limited API access and complex web scraping requirements demanded innovative solutions.

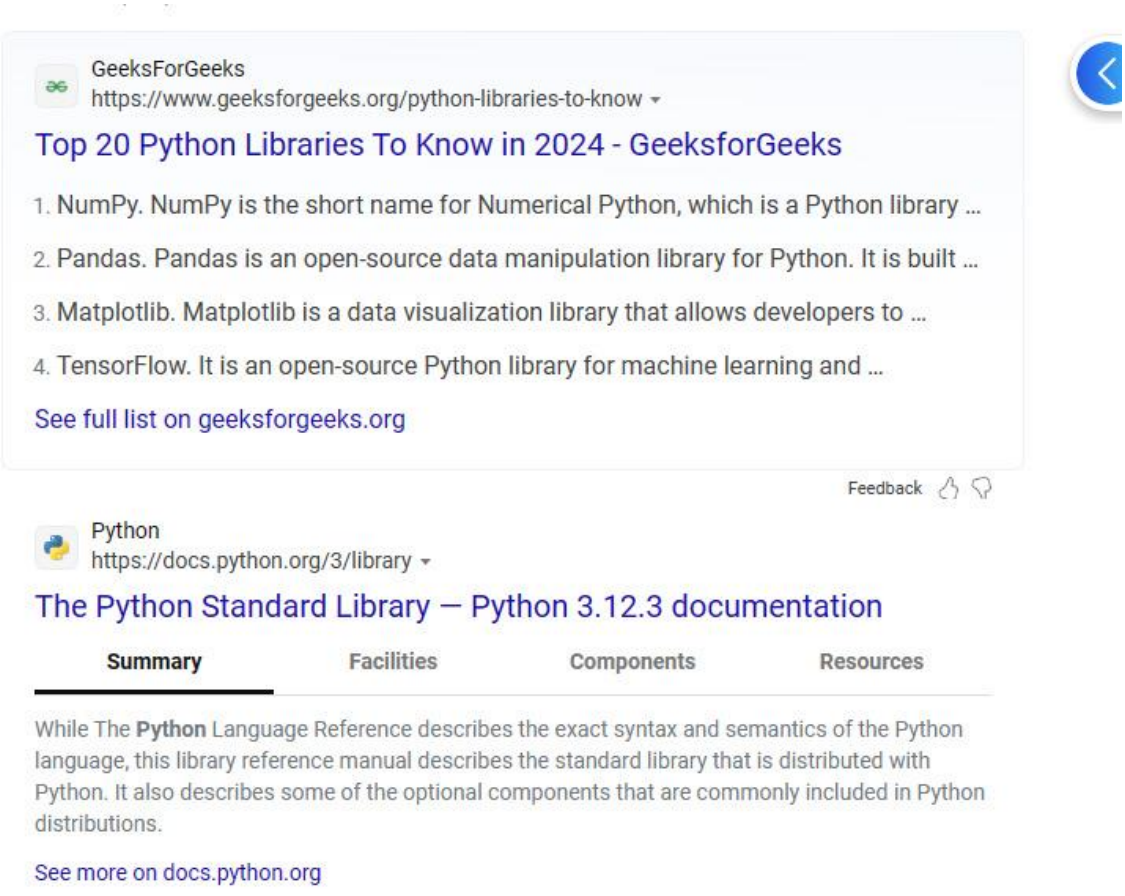
II. Data Preprocessing Complexity

The unstructured nature of scraped data necessitated intricate preprocessing techniques to extract meaningful information, such as titles, snippets, and URLs.

III. Arabic Language Processing

Implementing Arabic language support added layers of complexity due to unique text processing requirements, including text reshaping and bidirectional algorithms.

3 POINTS OF THE PROJECT



Stage 1: Requirements Gathering

Objective:to Define the scope and requirements of the search engine, including supported languages, targeted websites, and desired functionalities.

Approach: Conducted thorough research to identify suitable APIs and libraries for web scraping, language processing, and network analysis.

```
[90] import requests
     from bs4 import BeautifulSoup
     from serpapi import GoogleSearch

     params = {
         "q": "site:https://www.skynewsarabia.com اخبار",
         "engine": "google",
         "api_key": "707e1168809bb55fd9b3bb2219cb99e882b1ad050b943c3c0701ea5057f06a66",
         "gl":"eg"
     }

     search = GoogleSearch(params)
     results = search.get_dict()

     search_data = []
     for result in results["organic_results"]:
         search_data.append({
             "title": result["title"],
             "snippet": result["snippet"],
             "url": result["link"]
         })

     for result in results["organic_results"]:
         print(result["title"])
         print(result["snippet"])
         print(result["link"])
         print()
```

```
سكاي نيوز عربية: أخبار اليوم
ر الأخبار · حسام حسن: الكرة المصرية تعاني والدوري لا يقدم أي شيء · بشكل عفوي.. ملك بريطانيا يكتشف "سرا" عن حالته الصحية · إسرائيل تكثف القصف.. والبؤس يتعمق في
https://www.skynewsarabia.com/
```

Stage 2: Data Acquisition

Objective: Retrieve relevant data from targeted websites to populate the search engine index.

Approach: Utilized Python libraries such as Requests, BeautifulSoup, and serpapi to scrape search engine results pages (SERPs) from specified domains.

1

```
search = GoogleSearch(params)
results = search.get_dict()
search_data2 = []
for result in results["organic_results"]:
    search_data.append({
        "title": result["title"],
        "snippet": result["snippet"],
        "url": result["link"]
    })

for result in results["organic_results"]:
    print(result["title"])
    print(result["snippet"])
    print(result["link"])
```

2

```
[109] import matplotlib.pyplot as plt
      import arabic_reshaper
      from bidi.algorithm import get_display
      import networkx as nx

      G = nx.DiGraph()


      for result in search_data:
          title = get_display(arabic_reshaper.reshape(result["title"]))
          snippet = get_display(arabic_reshaper.reshape(result["snippet"]))
          G.add_node(title, type="title")
          G.add_node(snippet, type="snippet")
```

Stage 3: Data Preprocessing

Objective: Clean and preprocess scraped data to extract essential features such as titles, snippets, and URLs.

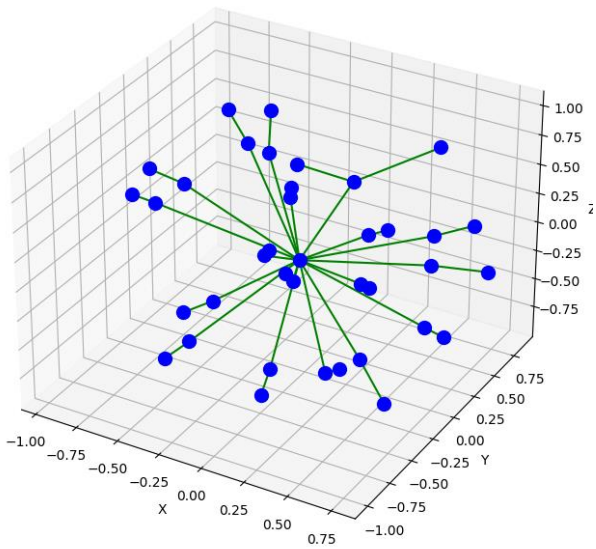
Approach: Implemented text reshaping and bidirectional algorithms to handle Arabic language text, ensuring proper display and processing.

```
[109] import matplotlib.pyplot as plt
      import arabic_reshaper
      from bidi.algorithm import get_display
      import networkx as nx

      G = nx.DiGraph()


      for result in search_data:
          title = get_display(arabic_reshaper.reshape(result["title"]))
          snippet = get_display(arabic_reshaper.reshape(result["snippet"]))
          G.add_node(title, type="title")
          G.add_node(snippet, type="snippet")

      for result in search_data:
          title = get_display(arabic_reshaper.reshape(result["title"]))
          snippet = get_display(arabic_reshaper.reshape(result["snippet"]))
          G.add_edge(title, snippet)

      main_node = get_display(arabic_reshaper.reshape("الترينـد"))
      G.add_node(main_node, type="main")

      for title_node in G.nodes():
          if G.nodes[title_node]["type"] == "title":
              G.add_edge(main_node, title_node)

      pos = nx.spring_layout(G, scale=1)

      plt.figure(figsize=(30, 17))

      nx.draw_networkx_nodes(G, pos, node_size=1000)
      nx.draw_networkx_edges(G, pos, width=3, alpha=0.5, edge_color='b')
      nx.draw_networkx_labels(G, pos, font_size=10, font_family="DejaVu Sans")

      plt.axis('off')
      plt.show()
```

Stage 4: Network Construction

Objective: Construct a network representation of search results, linking titles to snippets and main nodes.

Approach: Utilized NetworkX to create a directed graph, with nodes representing titles, snippets, and main categories, and edges representing relationships between them.




```
[114] import networkx as nx
      import matplotlib.pyplot as plt
      from mpl_toolkits.mplot3d import Axes3D

      pos = nx.spring_layout(G, dim=3)
      fig = plt.figure(figsize=(10, 8))
      ax = fig.add_subplot(111, projection='3d')
      for node, (x, y, z) in pos.items():
          ax.scatter(x, y, z, color='b', s=100)
      for edge in G.edges():
          u, v = edge
          x = [pos[u][0], pos[v][0]]
          y = [pos[u][1], pos[v][1]]
          z = [pos[u][2], pos[v][2]]
          ax.plot(x, y, z, color='g')
      ax.set_xlabel('X')
      ax.set_ylabel('Y')
      ax.set_zlabel('Z')
      plt.show()
```

Stage 4.2: Network Construction

Objective: Construct a network representation of search results, linking titles to snippets and main nodes.

Approach: Utilized NetworkX to create a directed graph, with nodes representing titles, snippets, and main categories, and edges representing relationships between them.

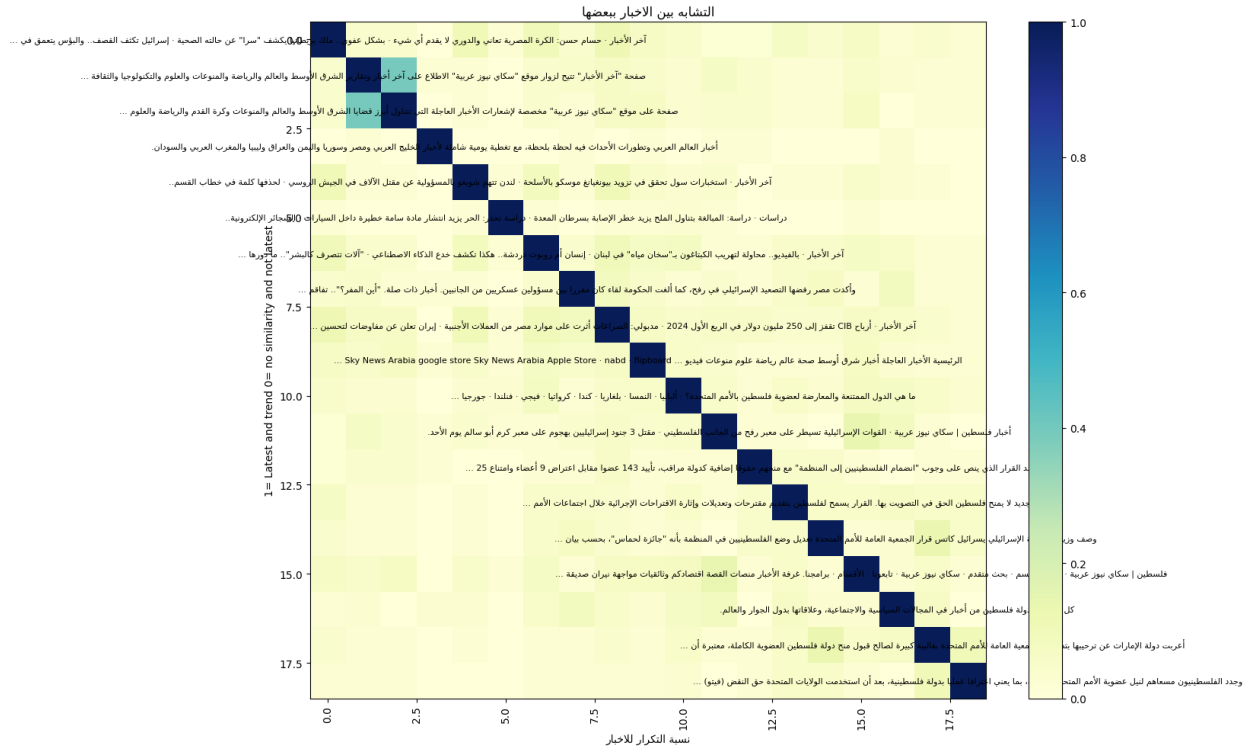
```
[110] degree_centrality = nx.degree_centrality(G)

      betweenness_centrality = nx.betweenness_centrality(G)

      print("node\tDegree Centrality\tBetweenness Centrality")

      for node in G.nodes():
          first_five_words = " ".join(node.split()[:5])
          degree_cent = degree_centrality.get(node, 0)
          betweenness_cent = betweenness_centrality.get(node, 0)
          print(f"{get_display(arabic_reshaper.reshape(first_five_words))}\t\t{degree_cent:.3f}\t\t\t{betweenness_cent:.3f}")

node    Degree Centrality       Betweenness Centrality
سكاي نيوز عربية: أخبار اليوم            0.054                   0.001
القصف.. والبؤس يتعمق في ...             0.027                   0.000
آخر الأخبار          0.054                   0.001
والمنوعات والعلوم والتكنولوجيا والثقافة ...             0.027                   0.000
الأخبار العاجلة                 0.054                   0.001
وكرة القدم والرياضة والعلوم ...         0.027                   0.000
أخبار الشرق الأوسط              0.054                   0.001
```

```
التريند              0.486                   0.000

[165] communities = list(nx.algorithms.community.greedy_modularity_communities(G))
      print("\nCommunities:")
      for i, community in enumerate(communities):
          print(f"Community {i + 1}: {community}")

      Communities:
      Community 1: frozenset({64, 1, 66, 67, 68, 6, 7, 72, 9, 74, 13, 79, 80, 17, 83, 84, 23, 25, 27, 32, 97, 99, 39, 43, 44, 45,
      Community 2: frozenset({0, 3, 4, 69, 70, 8, 11, 76, 14, 78, 16, 81, 82, 21, 22, 89, 26, 92, 29, 28, 93, 94, 31, 34, 33, 98,
      Community 3: frozenset({65, 5, 73, 10, 12, 77, 15, 18, 19, 20, 85, 87, 88, 91, 30, 95, 35, 36, 37, 38, 41, 42, 46, 49, 54,
      Community 4: frozenset({96, 2, 51, 52, 86, 24, 61})
      Community 5: frozenset({53, 90, 71, 58, 75})

      import networkx as nx
      import matplotlib.pyplot as plt
      from mpl_toolkits.mplot3d import Axes3D
```

Stage 5: Analysis and Visualization

Objective: Analyze the constructed network to derive insights into degree centrality, betweenness centrality, and community structures.

Approach: Leveraged NetworkX algorithms to compute centrality measures and community detection, visualizing the results using Matplotlib and 3D plotting libraries.



Generate a heatmap to visualize the similarity between search results, providing users with insights into the thematic clustering and relationships among different articles.

Approach:

- Similarity Matrix Calculation: Utilize word similarity measures to compute a similarity matrix between pairs of search results snippets.
- Heatmap Visualization: Employ Matplotlib to create a heatmap, with each cell representing the similarity score between two snippets.
- Labeling: Label the heatmap with snippets' content, providing users with context regarding the articles' content.
- Color Mapping: Apply a color gradient to represent the range of similarity scores, allowing users to discern closely related articles from those with lower similarity.